Lý thuyết và Chu trình Hoạt động của Static Hypernetwork cho Mạng Tích chập (CNN)

Phân tích Toán học Chi tiết và Ví dụ Tính toán Số học

Tài liệu Tổng hợp từ Công trình Nghiên cứu gốc
David Ha, Andrew Dai, Quoc V. Le (Google Brain, 2016)

Ngày 17 tháng 10 năm 2026

Tóm tắt nội dung

Tài liệu này trình bày chi tiết nền tảng lý thuyết, công thức toán học và chu trình hoạt động của mô hình **Static Hypernetwork** áp dụng cho Mạng Tích chập (Convolutional Neural Network - CNN), bám sát bài báo nghiên cứu *Hypernetworks* (Ha et al., 2016). Tài liệu bao gồm phân tích phân rã ma trận trọng số (weight factorization), cấu trúc 2 lớp tuyến tính dùng chung tham số, công thức so sánh độ phức tạp tham số, kỹ thuật ghép khối (tiling) cho các kiến trúc sâu (như ResNet), cùng một ví dụ tính toán số học từng bước cụ thể.

Mục lục

1 Giới thiệu Tổng quan **2**

2 Mô hình Toán học của Static Hypernetwork **2**
2.1 Cấu trúc Ma trận Kernel của Mạng CNN Chính 2
2.2 Kiến trúc 2 Lớp Tuyến tính của Hypernetwork 2

3 Chu trình Hoạt động (Working Cycle) **3**
3.1 Giai đoạn Huấn luyện (Training Phase) 3
3.2 Giai đoạn Suy luận (Inference Phase) 3

4 So sánh Số lượng Tham số (Parameter Complexity) **3**

5 Ví dụ Tính toán Số học Cụ thể (Step-by-Step Numerical Example) **3**
5.1 Đầu vào và Tham số Khởi tạo 4
5.2 Các Bước Tính Toán 4
5.2.1 Bước 1: Tính Vector Ẩn a_i^1 (Phép chiếu 1) 4
5.2.2 Bước 2: Tính Lát Kernel K_i^1 (Phép chiếu 2) 4
5.2.3 Bước 3: Ghép nối tạo Ma trận Kernel K_1 5

6 Kỹ thuật Ghép Khối (Tiling) cho Mạng Sâu **5**

7 Tài liệu Tham khảo **5**

1 Giới thiệu Tổng quan

Trong mạng tích chập sâu (Deep ConvNet), phần lớn các tham số có thể huấn luyện nằm ở ma trận kernel của các lớp tích chập. Khi mạng trở nên rất sâu, việc học độc lập từng trọng số dẫn đến sự dư thừa tham số nghiêm trọng. **Static Hypernetwork** là một mô hình mạng phụ (hypernetwork) nhỏ hơn, có nhiệm vụ "sinh"ra toàn bộ ma trận trọng số kernel cho mạng chính (main network). Thay vì lưu trữ trực tiếp hàng triệu trọng số, mạng chính chỉ cần lưu trữ một vector nhúng (layer embedding vector) z_j có kích thước siêu nhỏ cho mỗi lớp j. Chu trình này đại diện cho cơ chế chia sẻ trọng số nới lỏng (relaxed weight-sharing) giữa các lớp.

2 Mô hình Toán học của Static Hypernetwork

2.1 Cấu trúc Ma trận Kernel của Mạng CNN Chính

Xét một mạng CNN chính có độ sâu D lớp tích chập ($j = 1, \ldots, D$). Tại lớp tích chập thứ j:

- N_{in}: Số lượng kênh đầu vào (input channels).
- N_{out}: Số lượng kênh đầu ra (output channels).
- f_{size}: Kích thước không gian của bộ lọc 2D ($f_{\text{size}} \times f_{\text{size}}$).

Bài báo biểu diễn toàn bộ trọng số bộ lọc của lớp j dưới dạng một ma trận Kernel tổng K_j:

$$K_j \in \mathbb{R}^{(N_{\text{in}} \cdot f_{\text{size}}) \times (N_{\text{out}} \cdot f_{\text{size}})} \tag{1}$$

Ma trận K_j được phân rã thành N_{in} lát cắt ngang (slices) K_i^j, trong đó mỗi lát cắt ứng với kênh đầu vào thứ i ($i = 1, \ldots, N_{\text{in}}$):

$$K_i^j \in \mathbb{R}^{f_{\text{size}} \times (N_{\text{out}} \cdot f_{\text{size}})} \tag{2}$$

2.2 Kiến trúc 2 Lớp Tuyến tính của Hypernetwork

Hypernetwork $g(z_j)$ được thiết kế là một mạng tuyến tính 2 lớp (two-layer linear network) nhận đầu vào là vector nhúng lớp $z_j \in \mathbb{R}^{N_z}$:

Công thức Toán học Sinh Trọng số Kernel (Ha et al., 2016)

$$a_i^j = W_i z_j + B_i, \quad \forall i = 1, \ldots, N_{\text{in}}, \quad \forall j = 1, \ldots, D \tag{3}$$

$$K_i^j = \langle W_{\text{out}}, a_i^j \rangle + B_{\text{out}}, \quad \forall i = 1, \ldots, N_{\text{in}}, \quad \forall j = 1, \ldots, D \tag{4}$$

$$K_j = \begin{pmatrix} K_1^j \\ K_2^j \\ \vdots \\ K_{N_{\text{in}}}^j \end{pmatrix} \in \mathbb{R}^{(N_{\text{in}} \cdot f_{\text{size}}) \times (N_{\text{out}} \cdot f_{\text{size}})} \tag{5}$$

Trong đó:

- $z_j \in \mathbb{R}^{N_z}$: Vector nhúng tĩnh đại diện cho lớp j.
- $W_i \in \mathbb{R}^{d \times N_z}$ và $B_i \in \mathbb{R}^d$: Ma trận trọng số và bias của lớp thứ 1 thuộc Hypernetwork dành cho kênh đầu vào i, trong đó d là chiều ẩn (thường chọn $d = N_z$).
- $W_{\text{out}} \in \mathbb{R}^{f_{\text{size}} \times (N_{\text{out}} \cdot f_{\text{size}}) \times d}$: Tensor trọng số dùng chung ở lớp thứ 2 của Hypernetwork.

- $B_{\text{out}} \in \mathbb{R}^{f_{\text{size}} \times (N_{\text{out}} \cdot f_{\text{size}})}$: Ma trận bias dùng chung ở lớp thứ 2 của Hypernetwork.
- $\langle W_{\text{out}}, a_i^j \rangle$: Phép tích vô hướng tensor (tensor dot product) theo chiều ẩn d.

3 Chu trình Hoạt động (Working Cycle)

3.1 Giai đoạn Huấn luyện (Training Phase)

1. **Lan truyền tiến (Forward Pass):**
   - Nhận vector nhúng z_j của lớp j.
   - Tính vector ẩn a_i^j qua Lớp 1 (Phương trình 3).
   - Tính từng lát kernel K_i^j qua Lớp 2 (Phương trình 4).
   - Ghép N_{in} lát K_i^j tạo thành ma trận K_j (Phương trình 5).
   - Nắn dạng (reshape) K_j thành Tensor 4D $[N_{\text{out}}, N_{\text{in}}, f_{\text{size}}, f_{\text{size}}]$ để thực hiện phép tích chập `F.conv2d`(x, K_j) trên dữ liệu x.
   - Tính hàm mất mát (loss function) trên dữ liệu đầu ra của mạng chính.
2. **Lan truyền ngược (Backward Pass):**
   - Độ dốc (gradient) từ mạng chính được truyền ngược xuyên qua ma trận K_j về Hypernetwork.
   - Cập nhật đồng thời tất cả bộ tham số của Hypernetwork $\{W_i, B_i\}_{i=1}^{N_{\text{in}}}$, $W_{\text{out}}, B_{\text{out}}$ và toàn bộ các vector nhúng $\{z_j\}_{j=1}^{D}$ bằng thuật toán tối ưu hóa (như Adam hoặc SGD).

3.2 Giai đoạn Suy luận (Inference Phase)

Trong giai đoạn suy luận, mô hình chỉ cần nạp các vector nhúng z_j đã học và chạy qua Hypernetwork một lần để tái tạo lại ma trận K_j cho từng lớp j.

4 So sánh Số lượng Tham số (Parameter Complexity)

- **Mạng CNN truyền thống:** Tổng số tham số kernel của D lớp là:

$$P_{\text{CNN}} = D \cdot N_{\text{in}} \cdot N_{\text{out}} \cdot f_{\text{size}}^2 \tag{6}$$

- **Static Hypernetwork:** Tổng số tham số cần huấn luyện là:

$$P_{\text{Hyper}} = N_z \cdot D + d \cdot (N_z + 1) \cdot N_{\text{in}} + f_{\text{size}}^2 \cdot N_{\text{out}} \cdot (d + 1) \tag{7}$$

Nhờ việc chia sẻ bộ tham số W_{out} và B_{out} cho tất cả D lớp, khi D tăng lên, $P_{\text{Hyper}} \ll P_{\text{CNN}}$, giúp giảm số lượng tham số học được từ 80% đến 90%.

5 Ví dụ Tính toán Số học Cụ thể (Step-by-Step Numerical Example)

Xét một lớp tích chập với các thông số nhỏ sau:

- $N_{\text{in}} = 2$, $N_{\text{out}} = 2$, $f_{\text{size}} = 2$.
- Chiều vector nhúng $N_z = 2$, chiều ẩn $d = 2$.
- Xét lớp thứ $j = 1$.

5.1 Đầu vào và Tham số Khởi tạo

1. Vector nhúng của lớp $j = 1$:
$$z_1 = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix} \in \mathbb{R}^2$$

2. Tham số Lớp 1 thuộc Hypernetwork ($W_i \in \mathbb{R}^{2\times 2}, B_i \in \mathbb{R}^2$):
$$\text{Kênh } i = 1: \quad W_1 = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix}, \quad B_1 = \begin{pmatrix} 0.0 \\ 0.0 \end{pmatrix}$$
$$\text{Kênh } i = 2: \quad W_2 = \begin{pmatrix} 0.5 & 0.0 \\ 0.0 & 0.5 \end{pmatrix}, \quad B_2 = \begin{pmatrix} 0.1 \\ 0.2 \end{pmatrix}$$

3. Tham số Lớp 2 dùng chung ($W_{\text{out}} \in \mathbb{R}^{2\times 4\times 2}, B_{\text{out}} \in \mathbb{R}^{2\times 4}$):
Lát cắt K_i^1 có kích thước $f_{\text{size}} \times (N_{\text{out}} \cdot f_{\text{size}}) = 2 \times 4$. Ta tách W_{out} thành 2 ma trận 2×4 ứng với chiều $d = 2$:
$$W_{\text{out}}[:, :, 1] = \begin{pmatrix} 0.10 & 0.20 & 0.30 & 0.40 \\ 0.50 & 0.60 & 0.70 & 0.80 \end{pmatrix}$$
$$W_{\text{out}}[:, :, 2] = \begin{pmatrix} 0.20 & 0.10 & 0.40 & 0.30 \\ 0.60 & 0.50 & 0.80 & 0.70 \end{pmatrix}$$
$$B_{\text{out}} = \begin{pmatrix} 0.00 & 0.00 & 0.00 & 0.00 \\ 0.00 & 0.00 & 0.00 & 0.00 \end{pmatrix}$$

5.2 Các Bước Tính Toán

5.2.1 Bước 1: Tính Vector Ẩn a_i^1 (Phép chiếu 1)

$$a_1^1 = W_1 z_1 + B_1 = \begin{pmatrix} 1.0 & 0.0 \\ 0.0 & 1.0 \end{pmatrix} \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.0 \\ 0.0 \end{pmatrix} = \begin{pmatrix} 1.00 \\ 0.50 \end{pmatrix}$$
$$a_2^1 = W_2 z_1 + B_2 = \begin{pmatrix} 0.5 & 0.0 \\ 0.0 & 0.5 \end{pmatrix} \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0.1 \\ 0.2 \end{pmatrix} = \begin{pmatrix} 0.50 + 0.10 \\ 0.25 + 0.20 \end{pmatrix} = \begin{pmatrix} 0.60 \\ 0.45 \end{pmatrix}$$

5.2.2 Bước 2: Tính Lát Kernel K_i^1 (Phép chiếu 2)

Phép tích vô hướng tensor $\langle W_{\text{out}}, a_i^1 \rangle = a_i^1(1) \cdot W_{\text{out}}[:, :, 1] + a_i^1(2) \cdot W_{\text{out}}[:, :, 2]$:

- **Lát K_1^1 (ứng với Kênh vào 1):**
$$K_1^1 = 1.00 \begin{pmatrix} 0.10 & 0.20 & 0.30 & 0.40 \\ 0.50 & 0.60 & 0.70 & 0.80 \end{pmatrix} + 0.50 \begin{pmatrix} 0.20 & 0.10 & 0.40 & 0.30 \\ 0.60 & 0.50 & 0.80 & 0.70 \end{pmatrix}$$
$$= \begin{pmatrix} 0.10 + 0.10 & 0.20 + 0.05 & 0.30 + 0.20 & 0.40 + 0.15 \\ 0.50 + 0.30 & 0.60 + 0.25 & 0.70 + 0.40 & 0.80 + 0.35 \end{pmatrix}$$
$$= \begin{pmatrix} 0.200 & 0.250 & 0.500 & 0.550 \\ 0.800 & 0.850 & 1.100 & 1.150 \end{pmatrix}$$

- **Lát K_2^1 (ứng với Kênh vào 2):**

$$K_2^1 = 0.60 \begin{pmatrix} 0.10 & 0.20 & 0.30 & 0.40 \\ 0.50 & 0.60 & 0.70 & 0.80 \end{pmatrix} + 0.45 \begin{pmatrix} 0.20 & 0.10 & 0.40 & 0.30 \\ 0.60 & 0.50 & 0.80 & 0.70 \end{pmatrix}$$

$$= \begin{pmatrix} 0.060 + 0.090 & 0.120 + 0.045 & 0.180 + 0.180 & 0.240 + 0.135 \\ 0.300 + 0.270 & 0.360 + 0.225 & 0.420 + 0.360 & 0.480 + 0.315 \end{pmatrix}$$

$$= \begin{pmatrix} 0.150 & 0.165 & 0.360 & 0.375 \\ 0.570 & 0.585 & 0.780 & 0.795 \end{pmatrix}$$

5.2.3 Bước 3: Ghép nối tạo Ma trận Kernel K_1

Ghép K_1^1 và K_2^1 theo chiều dọc thu được ma trận $K_1 \in \mathbb{R}^{4\times4}$ chứa 16 trọng số kernel:

$$K_1 = \begin{pmatrix} K_1^1 \\ K_2^1 \end{pmatrix} = \begin{pmatrix} 0.200 & 0.250 & 0.500 & 0.550 \\ 0.800 & 0.850 & 1.100 & 1.150 \\ 0.150 & 0.165 & 0.360 & 0.375 \\ 0.570 & 0.585 & 0.780 & 0.795 \end{pmatrix}$$

6 Kỹ thuật Ghép Khối (Tiling) cho Mạng Sâu

Đối với các mạng như ResNet, số kênh $N_{\text{in}}, N_{\text{out}}$ thường là bội số của một kích thước cơ sở (ví dụ: $N_{\text{base}} = 16$). Để sinh ma trận kernel cho các lớp có $N_{\text{in}} = 32, N_{\text{out}} = 64$, Hypernetwork sinh ra 8 khối kernel cơ sở 16×16 riêng biệt từ 8 vector nhúng z khác nhau và ghép (tile) chúng lại:

$$K_{32\times64} = \begin{pmatrix} K_1 & K_2 & K_3 & K_4 \\ K_5 & K_6 & K_7 & K_8 \end{pmatrix} \tag{8}$$

7 Tài liệu Tham khảo

Tài liệu

[1] David Ha, Andrew Dai, Quoc V. Le. "Hypernetworks." *International Conference on Learning Representations (ICLR)*, 2017 (arXiv:1609.09106, 2016).